

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

Topic 4 – Async Research Assistant

Evidence from Wikipedia, arXiv and the web, collected concurrently and answered with citations

Research Assistant Team

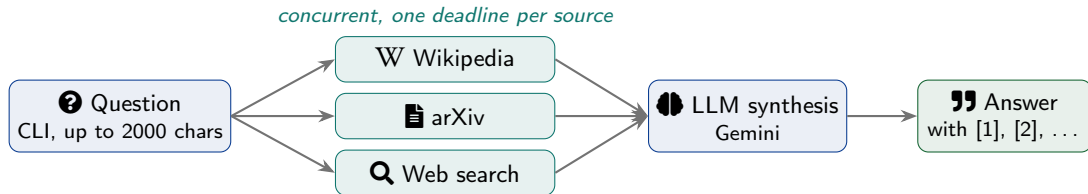
Ruslan Sadigov • Nihat Ismayilzade • Samur Eyyubov • Polad Ibrahimli

Final Project Defense – 20 September 2026

`github.com/ruslan-sadigov/research-assistant-SWE-Final-Project`

The problem

What the system does. A user asks a research question; the tool collects evidence from Wikipedia, arXiv and web search at the same time and prints an LLM answer whose numbered citations point to that evidence. A failing source becomes a warning, not a crash.



Inputs: the question (up to 2000 characters), `--sources` from `wiki`, `arxiv`, `web`, optional `--no-cache`; keys and limits from `.env`.

Outputs: answer, numbered references and per-source warnings; exit code 0 (answer), 1 (none) or 2 (invalid input).

Entry point: CLI only, no HTTP server: `python -m researcher ask "..."` `--sources` `wiki,arxiv,web`

Technology stack

BUILD & RUN



Runtime

Python 3.12 · asyncio

gather, timeout, to_thread



Environment

uv · uv.lock

Same versions everywhere



Config & models

Pydantic · pydantic-settings

Typed, self-validating

CONNECT & STORE



HTTP

httpx · RetryingTransport

Retries in one place



Storage

SQLite (sqlite3)

No server; survives restarts



Providers (ai/)

Wikipedia · arXiv · Tavily

+ Gemini; chosen by env vars

VERIFY & SHIP



Tests

pytest · asyncio · cov

Offline, async, measured



Quality gates

Ruff · Black · isort · mypy

One style, typed signatures



Delivery

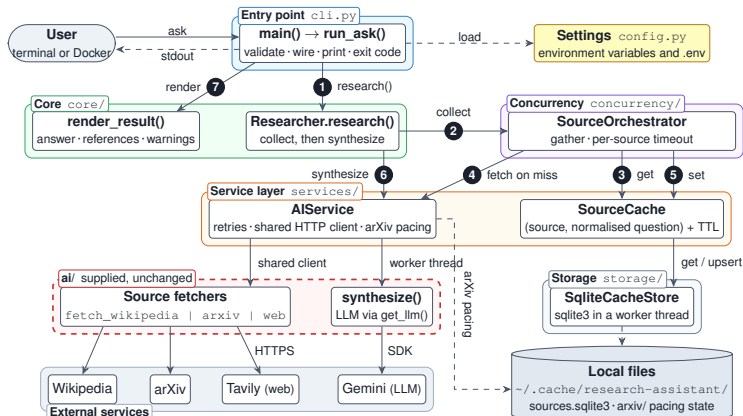
GitHub Actions · Docker

Lint, types, tests per push

Everything except the supplied ai/ package and its providers is our own code and configuration.

Architecture

One request, five layers. The supplied `ai/` package (dashed) is called only from `AIService` and was not modified.



Solid arrows: calls, numbered in request order. Dashed: configuration, local files, output.

Key design decisions

1

asyncio over threads, one deadline per source

All the work is waiting on at most three sources, so one event loop is enough.

Rejected: threads, and TaskGroup, which cancels the other sources when one fails.

2

Protocols and constructor injection around the unchanged ai/

Tests swap in fakes without patching, and mypy still checks the signatures.

Trade-off accepted: one wiring function (run_ask) must build everything in order.

3

SQLite cache of source evidence, not of answers

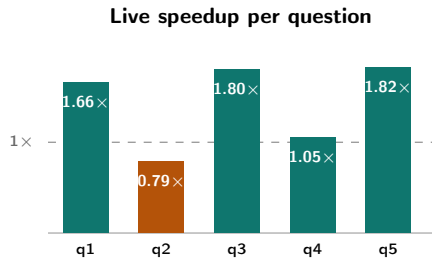
Every run still synthesizes; repeated questions skip the network.

We would change it for many parallel users (single-file writer) or if answers became expensive.

Concurrency: sequential vs. concurrent

Workload: the 5 supplied questions $\times$ all 3 sources, cache off. Offline: simulated delays 0.1 / 0.2 / 0.3 s, median of 5 runs. Live: real Wikipedia, arXiv and Tavily requests, one pair per question.

	Sequential	Concurrent
Offline wall-clock (s)	0.623–0.628	0.312–0.314
Offline speedup	1.0 $\times$	1.99–2.00 $\times$
Live wall-clock (s)	4.41–7.04	3.62–5.60
Live speedup (q1–q5)	1.0 $\times$	0.79–1.82 $\times$
Provider 429s observed	0	0
Failed or timed-out sources	0	0



Bottleneck: the slowest source sets the concurrent time (0.3 s offline); live, network latency and arXiv's 3 s request spacing.

Why not 3 $\times$: with unequal delays the ideal is 2 $\times$ (sum / max = 0.6 / 0.3); live, q2 was slower concurrently and Wikipedia found nothing for full questions.

Reproduce: `uv run python benchmarks/benchmark_sources.py --repeats 5`

Robustness: when a source fails

Scenario: in a live run with all three sources, arXiv timed out. **Found by:** a manual live CLI run, reproduced offline in `test_deadline_cancels_slow_fetch_and_preserves_success`.

Retry



Up to 3 attempts, 0.5s growing to 4s; only timeouts, 408, 429 and 5xx are retried; Retry-After is honoured; one 10s deadline per source.

Validate



Input and settings are checked before any call (exit 2); models reject unknown fields; cached rows are re-validated on read.

Log



source timeout: source=arxiv stage=... Failures log the error type only, never keys or messages.

Degrade



Wikipedia's evidence was kept and the answer was printed with an arXiv warning; exit code 0.

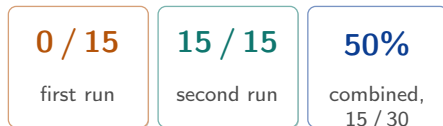
Also seen live: intermittent Gemini 503 (high demand); later calls succeeded.

Caching: measured, not assumed

Schema and policy

- ▶ Table `cache_entries`: key (source, query_key), its automatic unique index, JSON payload; no foreign keys
- ▶ Key = source + normalised question; TTL 24 h
- ▶ All rows derived; the external sources stay authoritative
- ▶ `--no-cache` never opens the SQLite file

Hit rate = hits / lookups



5 questions $\times$ 3 sources through the real CLI; providers simulated; an answer is synthesized on every run.

Staleness: empty results are also kept for 24 h, and the key ignores provider settings.

Reproduce: `uv run python tests/cache_verification.py` (*docs/cache-verification.md*)

What we will run

One question against all three sources, twice, with a cache volume: the second run is served from SQLite and still produces an answer.

Command (runs inside Docker):

```
docker run --rm --env-file .env --env DATABASE_URL= `
-v research-assistant-cache:/home/appuser/.cache/research-assistant `
research-assistant:runtime ask "Photovoltaic effect" --sources wiki,arxiv,web
```

- ▶ On screen: numbered references from all three sources; on the second run, `cache_hit=True` in the log.
- ▶ Timing: in a live container run the sources took 2.39, 3.53 and 0.53 s, yet collection finished in 3.64 s instead of 6.45 s.

Testing

269

own tests

16

provided smoke
tests, passing

98.58 %

coverage of
src/researcher

0

mypy issues in
15 source files

Key tests

- ▶ **Happy path:** the real CLI answers each supplied question, reuses SQLite, then honours `--no-cache`.
- ▶ **Concurrency:** arXiv raises, gather degrades; Wikipedia and web evidence are kept.
- ▶ **Error path:** HTTP 408/429/500/503 is retried, then succeeds; every response is closed.

Tooling

pytest + pytest-asyncio + pytest-cov + httpx.MockTransport; all tests run offline with the AI module mocked. Ruff, Black and isort are clean in CI.

Limitations & next steps

Where the system is brittle

- ▶ Full questions often find no Wikipedia match; there is no query rewriting.
- ▶ No fallback LLM: if Gemini is down, we show evidence but no answer.
- ▶ The cache keeps empty results for 24 h, ignores provider settings, and never evicts rows.

With another week

1. Turn the question into a short topic query before searching Wikipedia.
2. Add a settings fingerprint to the cache key, a shorter TTL for empty results, and a purge command.
3. Repeat the live benchmark at least three times and compare distributions.

Team contributions

Member	Primary ownership	Commits
Ruslan Sadigov	Settings, models, interfaces, orchestrator, retry transport, arXiv pacing; merges and release	 38 (50.0%)
Nihat Ismayilzade	AIService around ai/; engineering audit, tests, Docker and provider docs	 10 (13.2%)
Samur Eyyubov	SourceCache and SQLite store; architecture diagram, cache verification, slides, demo	 15 (19.7%)
Polad Ibrahimli	CLI and Researcher; report, README, contribution statement	 13 (17.1%)

Non-merge commits on main, 20 Sep 2026: 76 in total, author aliases merged. Per-file ownership: signed contribution statement.

AI assistance

Claude and OpenAI Codex were used; each module's use, and what we changed, is disclosed in report §9 and in the signed contribution statement.

Questions?

Research Assistant Team

Ruslan Sadigov • Nihat Ismayilzade • Samur Eyyubov • Polad Ibrahimli

`github.com/ruslan-sadigov/research-assistant-SWE-Final-Project`

Thank you.